

INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC



CARRERA DESARROLLO DE SOFTWARE

DEVOPS

Gestión de Repositorios Colaborativos, Protección de Ramas y Automatización CI/CD en GitHub

Nombres completos: BRAYAN STEVEN NINASUNTA QUINDIL

BRYAN ESTEVEN VIRACOCOA YAGUAPAZ

TUTOR: Byron Moreno

QUITO – ECUADOR

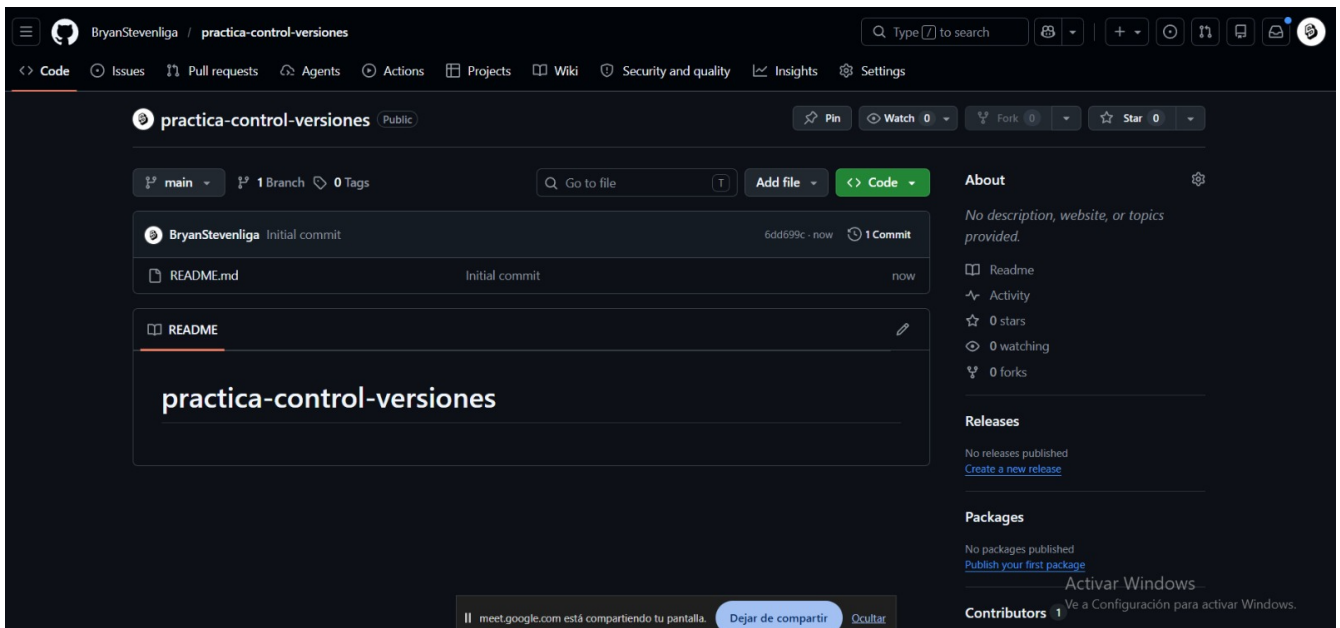
JUNIO, 2026

Trabajo Colaborativo en Parejas

Inicialización del Repositorio Remoto en GitHub

1. **Actividad Realizada:** Se creó el repositorio remoto en GitHub denominado practica-control-versiones con configuración Pública, garantizando el acceso para la evaluación docente. Durante este proceso, se inicializó el entorno con la creación automática del archivo base README.md.

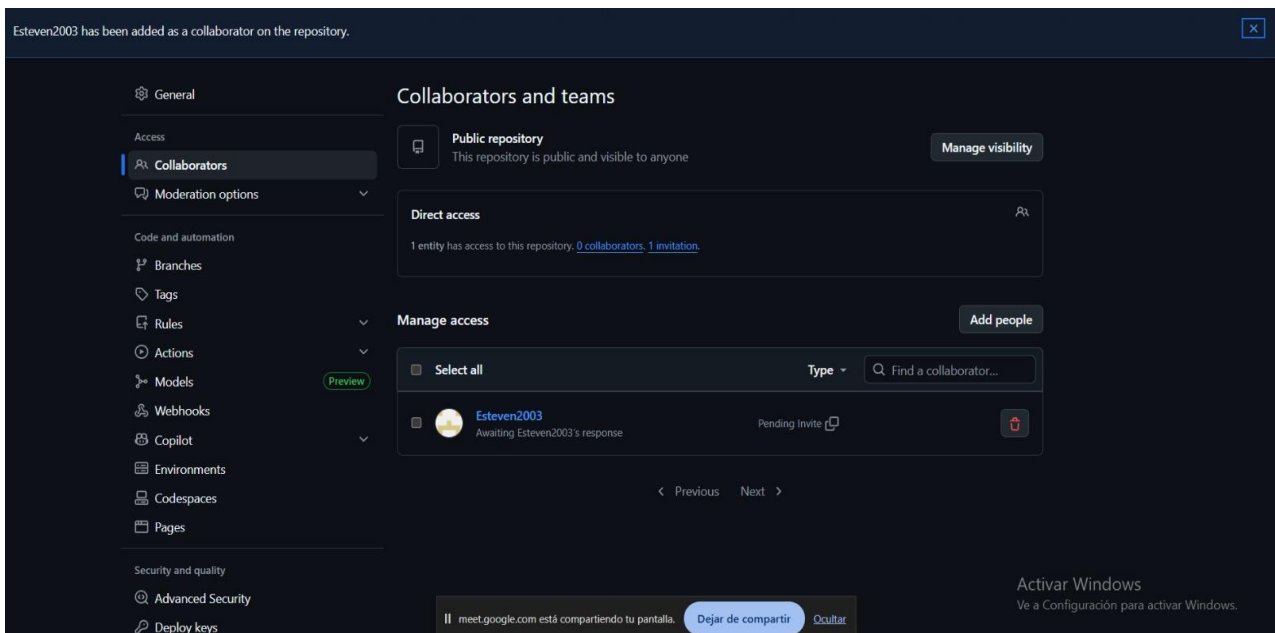
Resultado Obtenido: Como se observa en la captura, el repositorio quedó desplegado de forma exitosa en la nube con un único commit inicial (Initial commit) y configurado sobre la rama principal main, quedando listo para la fase de clonación y desarrollo local.



Configuración de Colaboradores en GitHub

2. **Actividad Realizada:** El Creador del repositorio accedió a los ajustes de administración (Settings -> Collaborators) e invitó formalmente al usuario Esteven2003 (Estudiante B) para integrarlo como desarrollador colaborador del proyecto.

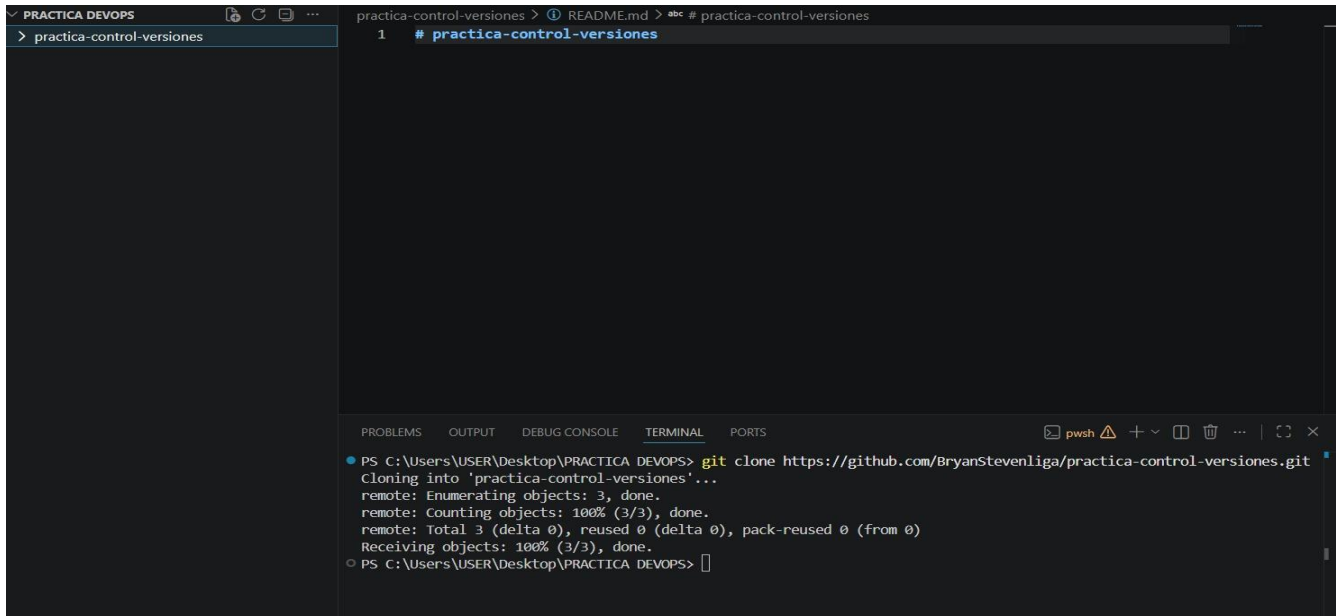
Resultado Obtenido: Como se evidencia en la captura de pantalla, el sistema envió y registró la invitación de forma correcta, quedando en estado pendiente (Pending Invite) a la espera de la aceptación por parte del compañero para iniciar el trabajo colaborativo.



Clonación del Repositorio Remoto al Entorno Local

3. **Actividad Realizada:** Se procedió a descargar una copia exacta del repositorio remoto mediante el uso del editor Visual Studio Code. Para esto, se abrió la terminal integrada y se ejecutó el comando técnico git clone seguido de la URL del proyecto hospedado en GitHub.

Resultado Obtenido: Como se observa en la captura de pantalla, el proceso finalizó con éxito (Receiving objects: 100%), descargando el archivo base README.md y vinculando el directorio local directamente con el repositorio en la nube para iniciar el desarrollo.



The screenshot shows a Visual Studio Code interface with a terminal window open. The terminal displays the output of a git clone command. The command executed is `git clone https://github.com/BryanStevenliga/practica-control-versiones.git`. The output shows the cloning process, including enumerating objects, counting objects, and receiving objects, all of which completed successfully at 100%.

```
practica-control-versiones > README.md > abc # practica-control-versiones
1 # practica-control-versiones

PS C:\Users\USER\Desktop\PRACTICA DEVOPS> git clone https://github.com/BryanStevenliga/practica-control-versiones.git
Cloning into 'practica-control-versiones'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
PS C:\Users\USER\Desktop\PRACTICA DEVOPS>
```

Clonación del Repositorio Remoto por el Estudiante B y Verificación de Remotos

4. Actividad Realizada: El Estudiante B (usuario Esteven1) ejecutó en su terminal local el comando `git clone` para descargar el repositorio compartido. Acto seguido, usó el comando `cd` para ingresar al directorio del proyecto y ejecutó `git remote -v` para listar y verificar las direcciones URL remotas de lectura (fetch) y escritura (push).

Resultado Obtenido: Como se evidencia en la captura de pantalla, la descarga de objetos finalizó correctamente al 100%. Además, la terminal confirmó la correcta vinculación con el repositorio central origin de GitHub, quedando el entorno de desarrollo del colaborador listo para iniciar el flujo de trabajo en equipo.

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base: main ← compare: feature-html ✓ Able to merge. These branches can be automatically merged.

Add a title *

Agregar estructura inicial de html e index

Add a description

Write Preview H B I | | ...

* **Descripción (Leave a comment):** Copia y pega esta estructura limpia (puedes adaptarla ligeramente):

```
``markdown
## Descripción del Cambio

Se ha creado el archivo base `index.html` con la estructura fundamental de HTML5 para dar inicio al proyecto colaborativo.

## Tareas Realizadas
- [x] Creación del archivo `index.html`.
- [x] Inclusión de etiquetas `<!DOCTYPE html>`, `<head>`, y `<body>`.
- [x] Adición de un título `<h1>` y un párrafo principal con ID para modificaciones futuras.

## Evidencia

La rama local `feature-html` se compiló correctamente y no presenta errores sintácticos.

Asignado a: @[BryanStevenliga]
```

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

Markdown is supported

Paste, drop, or click to add files

Create pull request

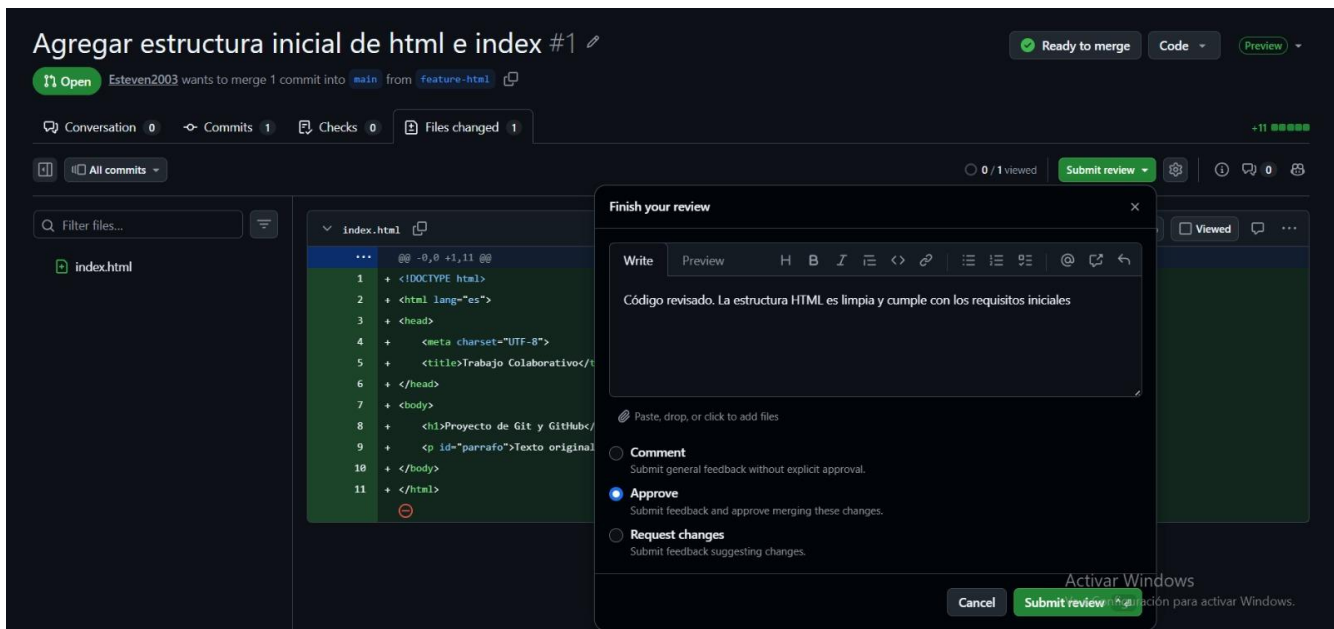
Registro de Cambios Locales y Publicación de la Rama al Repositorio Remoto

(Estudiante B)

5. **Actividad Realizada:** El Estudiante B preparó el archivo modificado usando el comando `git add index.html` y registró los cambios localmente en su rama de trabajo mediante `git commit`, incluyendo un mensaje descriptivo y estandarizado ("Feat: Agregar

estructura inicial"). Posteriormente, ejecutó el comando `git push origin feature-html` para subir la rama y los commits a la nube.

Resultado Obtenido: Como se observa en la captura de pantalla, Git procesó con éxito la subida de los datos (Writing objects: 100%) y creó de manera efectiva la nueva rama remota `feature-html` en el servidor de GitHub, generando automáticamente el enlace directo en la terminal para proceder con la apertura del correspondiente Pull Request.



Apertura del Pull Request en GitHub (Estudiante B)

6. **Actividad Realizada:** Tras publicar los cambios en la nube, el Estudiante B ingresó a la interfaz de GitHub para iniciar el proceso de integración del código. Configuró el Pull Request para comparar la rama de trabajo `feature-html` contra la rama principal `base: main` y le asignó un título formal y descriptivo: "Feat: Agregar estructura inicial".

Resultado Obtenido: Como se observa en la captura de pantalla, la plataforma validó automáticamente que no existen conflictos previos entre las ramas mostrando el

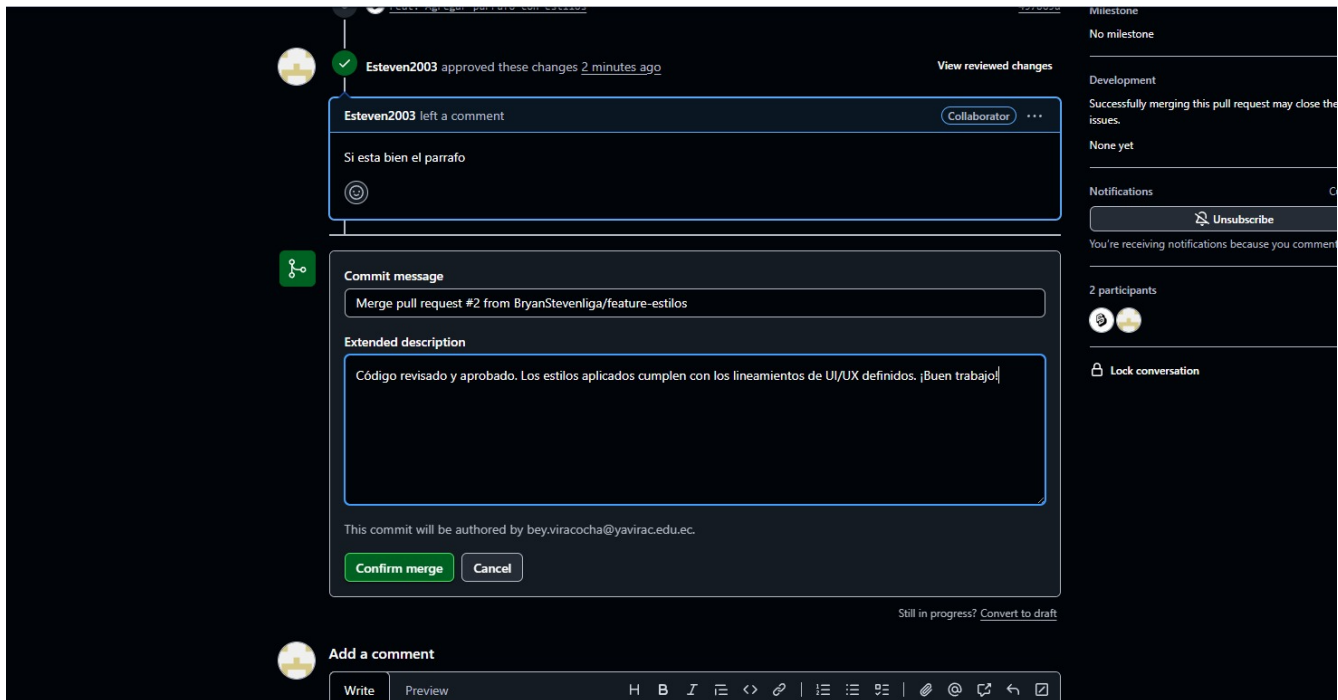
mensaje en verde Able to merge. El formulario quedó completado y listo para que el colaborador presionara el botón Create pull request, dejando la propuesta a disposición del revisor.

```
PS C:\Users\Esteven1\practica_versiones_control> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\Esteven1\practica_versiones_control> git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 926 bytes | 185.00 KiB/s, done.
From https://github.com/BryanStevenliga/practica_versiones_control
* branch          main          -> FETCH_HEAD
   dc65c9b..78c1675 main        -> origin/main
Updating dc65c9b..78c1675
Fast-forward
 index.html | 11 ++++++++
 1 file changed, 11 insertions(+)
 create mode 100644 index.html
PS C:\Users\Esteven1\practica_versiones_control> █
```

Revisión y Aprobación del Código en el Pull Request (Estudiante A)

7. **Actividad Realizada:** El Estudiante A (Creador del repositorio) ingresó a la pestaña Files changed para auditar los cambios propuestos por su compañero. Tras validar la sintaxis, abrió el módulo Submit review, escribió un comentario de validación profesional ("Código revisado. La estructura HTML es limpia y cumple con los requisitos iniciales") y seleccionó explícitamente la opción de estado Approve (Aprobar).

Resultado Obtenido: Como se evidencia en la captura de pantalla, el sistema dejó listo el formulario de revisión para ser enviado mediante el botón Submit review, otorgando el visto bueno técnico necesario y de carácter obligatorio para liberar la posterior fusión de ramas.



Sincronización del Repositorio Local con los Cambios Fusionados en Remoto (Estudiante B)

8. **Actividad Realizada:** El Estudiante B regresó a la rama principal de su entorno local usando el comando `git checkout main`. Posteriormente, con el fin de bajarse la estructura ya aprobada y fusionada en la nube por su compañero, ejecutó el comando de sincronización `git pull origin main`.

Resultado Obtenido: Como se observa en la captura de pantalla, Git realizó una integración limpia de tipo Fast-forward, trayendo de manera exitosa el nuevo archivo `index.html` con sus correspondientes 11 líneas agregadas (1 file changed, 11 insertions(+)), lo que garantiza que ambos entornos locales queden alineados con la última versión del servidor remoto.

```
PS C:\Users\Esteven1\practica versiones control> git pull origin main
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1.25 KiB | 53.00 KiB/s, done.
From https://github.com/BryanStevenliga/practica_versiones_control
* branch      main      -> FETCH_HEAD
   256b940..286748f  main      -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Esteven1\practica_versiones_control> 
```

Creación de Rama de Trabajo, Commit y Publicación de Cambios (Estudiante

A)

9. **Actividad Realizada:** Para dar cumplimiento al intercambio de roles, el Estudiante A creó una nueva rama local mediante el comando `git checkout -b feature-estilos` para añadir nuevas funcionalidades de diseño. Tras modificar el código, preparó el archivo con `git add index.html`, registró el cambio localmente usando `git commit -m "Feat: Agregar parrafo con estilos"` y finalmente publicó la rama en el servidor remoto con `git push origin feature-estilos`.

Resultado Obtenido: Como se observa en la captura de pantalla, Git procesó y subió los cambios con éxito al repositorio remoto (Writing objects: 100%), desplegando de forma efectiva la nueva rama `feature-estilos` en GitHub y habilitando el enlace correspondiente para gestionar el nuevo Pull Request.

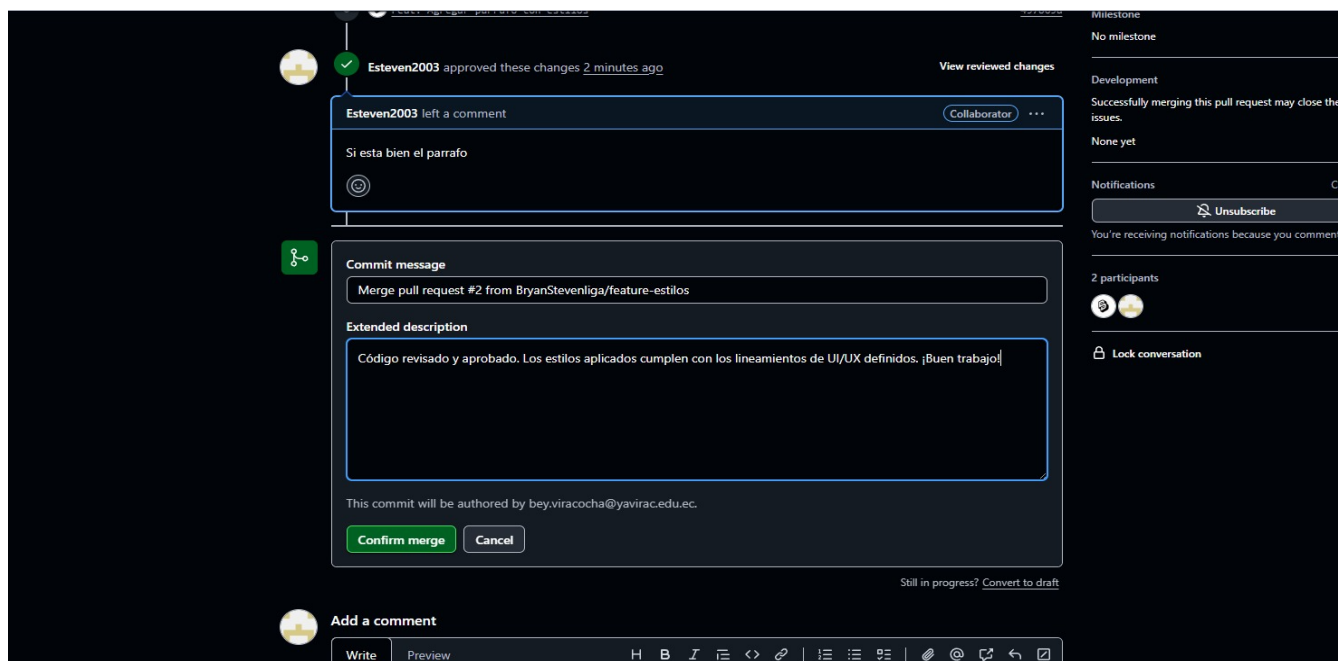
```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  POSTMAN CONSOLE

PS C:\Users\Esteven1\practica_versiones_control> git pull origin main
* branch          main      -> FETCH_HEAD
256b940..286748f  main      -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
● PS C:\Users\Esteven1\practica_versiones_control> git add index.html
● PS C:\Users\Esteven1\practica_versiones_control> git commit -m "Fix: Resolviendo conflicto de integracion"
[conflicto-B 671d6ff] Fix: Resolviendo conflicto de integracion
● PS C:\Users\Esteven1\practica_versiones_control> git push origin conflicto-B
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 427 bytes | 213.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/BryanStevenliga/practica_versiones_control.git
c3a6233..671d6ff  conflicto-B -> conflicto-B
○ PS C:\Users\Esteven1\practica_versiones_control> █
```

Revisión por Colaborador y Confirmación de la Fusión de Ramas (Merge) en GitHub

10. **Actividad Realizada:** El Estudiante B (Esteven2003) auditó de forma cruzada la propuesta de cambios de la rama feature-estilos, emitiendo una aprobación formal acompañada del comentario "Si esta bien el parrafo". Seguidamente, se procedió a la consolidación técnica del segundo Pull Request (#2), preparando la integración final con una descripción extendida para registrar adecuadamente el evento en el historial del proyecto.

Resultado Obtenido: Como se constata en la captura de pantalla, el entorno web de GitHub validó la aprobación técnica del colaborador y habilitó el botón verde de acción Confirm merge. Esto dejó el flujo listo para cerrar el ciclo de desarrollo colaborativo de la nueva característica e incorporar los estilos directamente en la rama central main.

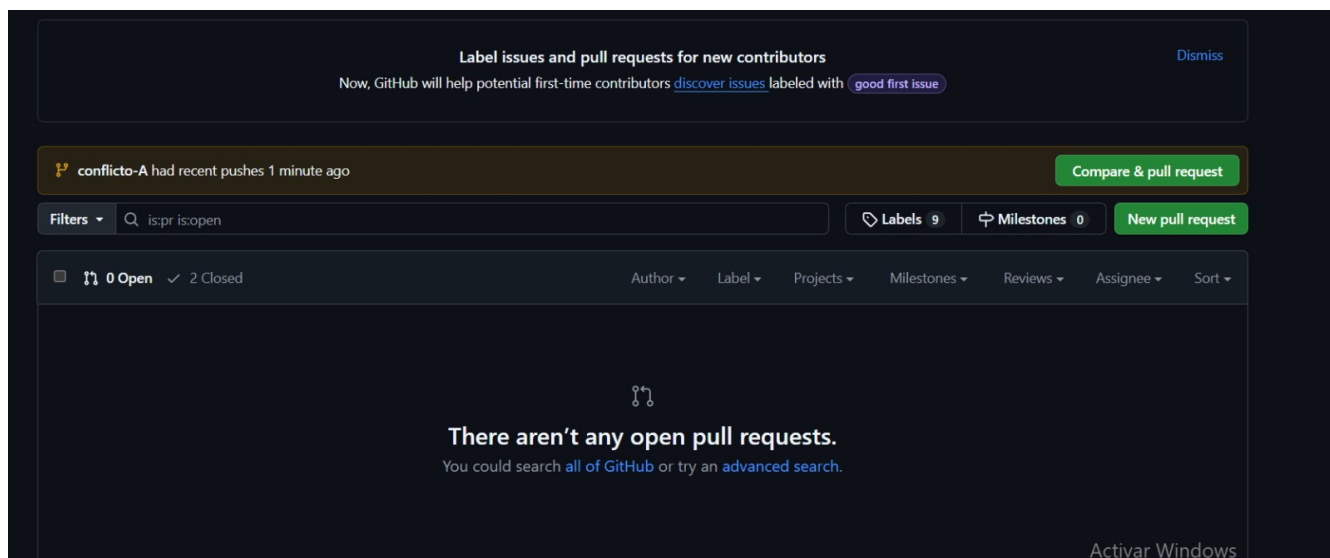


Generación de una Rama de Trabajo para la Simulación de Conflictos

(Estudiante A)

11. **Actividad Realizada:** Con el objetivo de simular y documentar un conflicto de fusión en un escenario real de integración de software, el Estudiante A subió una nueva rama de trabajo modificando de forma paralela las mismas líneas de código. Al ingresar a la interfaz de GitHub, la plataforma reconoció el evento de actualización bajo la rama denominada conflicto-A.

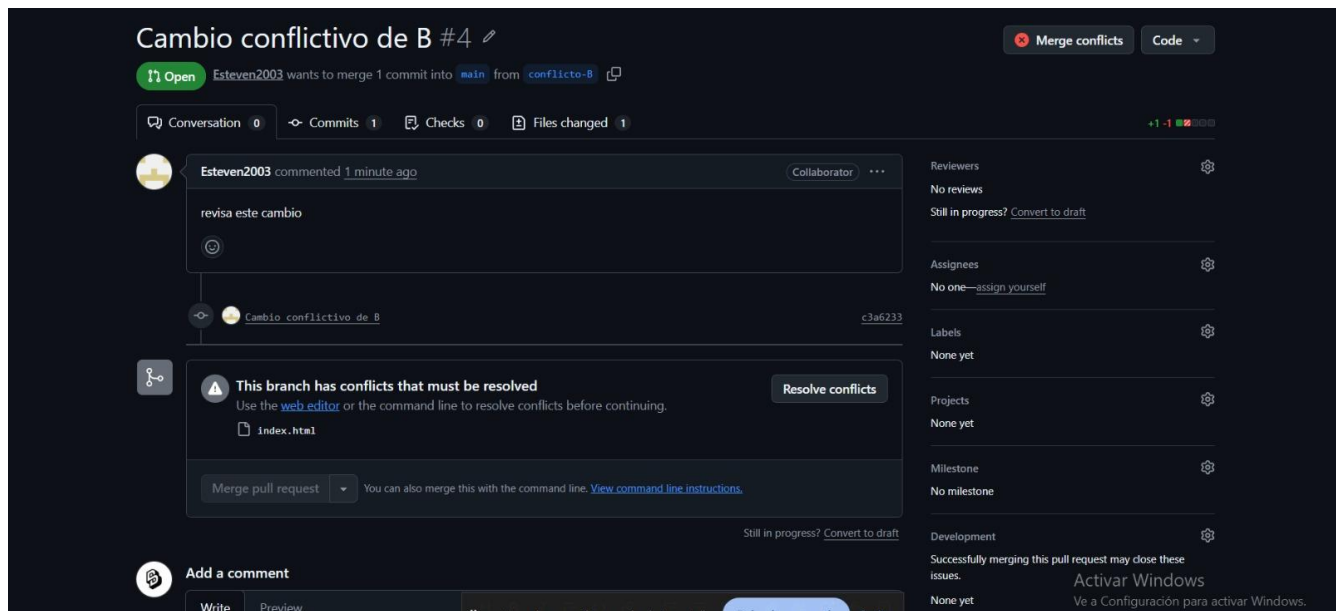
Resultado Obtenido: Como se observa en la captura de pantalla el sistema desplegó de forma correcta la alerta dorada que indica un push reciente (conflicto-A had recent pushes 1 minute ago) junto al botón verde de acción Compare & pull request, dando inicio técnico a la etapa de provocación y resolución guiada de colisiones en el código.



Apertura de Pull Request en Conflicto e Identificación del Bloque Afectado (Estudiante B)

12. **Actividad Realizada:** El Estudiante B (Esteven2003) procedió a abrir el Pull Request número 4 (#4) titulado "Cambio conflictivo de B" desde su rama conflicto-B hacia la rama principal, añadiendo el comentario "revisa este cambio". Al intentar integrarlo de manera directa, el sistema comparó de forma automatizada los bloques de código concurrentes.

Resultado Obtenido: Como se evidencia de forma clara en la captura de pantalla, GitHub bloqueó automáticamente la fusión de cambios arrojando la alerta crítica de estado: This branch has conflicts that must be resolved. El software identificó explícitamente al archivo index.html como la fuente de la colisión, inhabilitando el botón de merge y mostrando la opción Resolve conflicts para obligar al equipo a realizar la corrección manual.



Manifestación del Conflicto de Fusión en la Terminal Local (Estudiante B)

13. **Actividad Realizada:** Para gestionar y solucionar el problema desde el entorno de desarrollo local, el Estudiante B (Esteven1) ejecutó el comando `git pull origin main` en su consola con el fin de descargar la última versión del repositorio e integrarla con sus cambios pendientes.

Resultado Obtenido: Como se observa detalladamente en la captura de pantalla, el motor de Git detuvo el proceso automático y arrojó la advertencia técnica: **CONFLICT (content): Merge conflict in index.html**. La terminal desplegó formalmente el aviso **Automatic merge failed; fix conflicts and then commit the result**, dejando el flujo de trabajo suspendido hasta que se edite el archivo y se elijan de forma manual las líneas de código definitivas.

```
PS C:\Users\Esteven1\practica versiones control> git pull origin main
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 4 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1.25 KiB | 53.00 KiB/s, done.
From https://github.com/BryanStevenliga/practica_versiones_control
* branch      main      -> FETCH_HEAD
   256b940..286748f  main      -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Esteven1\practica_versiones_control> 
```

Resolución Manual del Conflicto y Publicación de la Solución (Estudiante B)

14. **Actividad Realizada:** Posterior a la detección del error de contenido en index.html, el Estudiante B editó el código de manera manual para integrar de forma armónica ambas propuestas. Una vez corregido, ejecutó el comando `git add index.html` para marcar el conflicto como resuelto, seguido de `git commit -m "Fix: Resolviendo conflicto de integracion"` para asentar el commit de fusión, y finalizó enviando los cambios corregidos mediante `git push origin conflicto-B`.

Resultado Obtenido: Como se evidencia en la captura de pantalla, Git procesó y empaquetó con éxito la solución definitiva (Writing objects: 100%), actualizando la rama remota en GitHub con el nuevo hash de confirmación (671d6ff) y dejando el Pull Request libre de colisiones para su cierre definitivo.

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  POSTMAN CONSOLE

PS C:\Users\Esteven1\practica_versiones_control> git pull origin main
* branch          main          -> FETCH_HEAD
   256b940..286748f  main          -> origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
● PS C:\Users\Esteven1\practica_versiones_control> git add index.html
● PS C:\Users\Esteven1\practica_versiones_control> git commit -m "Fix: Resolviendo conflicto de integracion"
[conflicto-B 671d6ff] Fix: Resolviendo conflicto de integracion
● PS C:\Users\Esteven1\practica_versiones_control> git push origin conflicto-B
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 427 bytes | 213.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/BryanStevenliga/practica_versiones_control.git
   c3a6233..671d6ff  conflicto-B -> conflicto-B
○ PS C:\Users\Esteven1\practica_versiones_control> █
```

Validación y Habilitación de la Fusión tras Resolver Conflictos

15. **Actividad Realizada:** Se verificó en la interfaz web de GitHub el estado del Pull Request número 4 (#4) luego de que el colaborador subiera el commit de solución (671d6ff). La plataforma procesó de manera automática los dos commits de la rama conflicto-B.

Resultado Obtenido: Como se detalla en la captura de pantalla , el sistema validó la corrección manual cambiando el estado a verde con el mensaje No conflicts with base branch. Al quedar libre de colisiones, se desbloqueó el botón de acción Merge pull request, habilitando la integración limpia y definitiva de los cambios hacia la rama main.

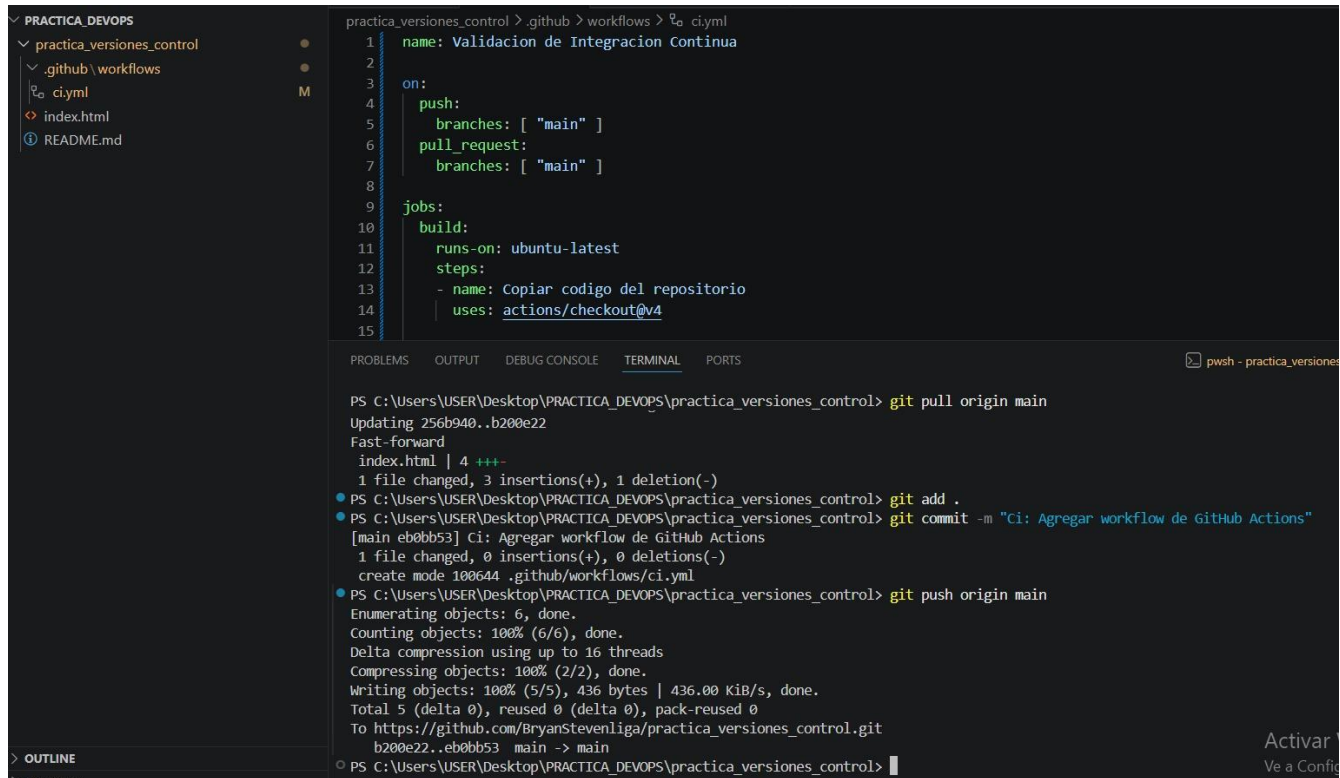


Configuración e Implementación de un Flujo de Integración Continua (GitHub Actions)

16. **Actividad Realizada:** Se creó el archivo de automatización ci.yml dentro del directorio .github/workflows con las reglas de ejecución para los eventos push y pull_request en la rama main. Tras alinear el entorno con git pull origin main, se preparó la configuración usando git add ., se registró el cambio mediante git commit -m "Ci: Agregar workflow de GitHub Actions" y se publicó con git push origin main.

Resultado Obtenido: Como se detalla en la captura de pantalla, el archivo de Integración Continua denominado "Validacion de Integracion Continua" se subió con éxito al servidor remoto (Writing objects: 100%), dejando el repositorio listo para disparar tareas

automáticas en un entorno virtualizado de Linux (ubuntu-latest) cada vez que se actualice el código.



The screenshot shows a VS Code editor with a file explorer on the left displaying the project structure: `PRACTICA_DEVOPS`, `practica_versiones_control`, `.github/workflows`, `ci.yml`, `index.html`, and `README.md`. The main editor area shows the content of `ci.yml`:

```
1 name: Validacion de Integracion Continua
2
3 on:
4   push:
5     branches: [ "main" ]
6   pull_request:
7     branches: [ "main" ]
8
9 jobs:
10  build:
11    runs-on: ubuntu-latest
12    steps:
13      - name: Copiar codigo del repositorio
14        uses: actions/checkout@v4
15
```

Below the editor, the `TERMINAL` tab is active, showing the following commands and output:

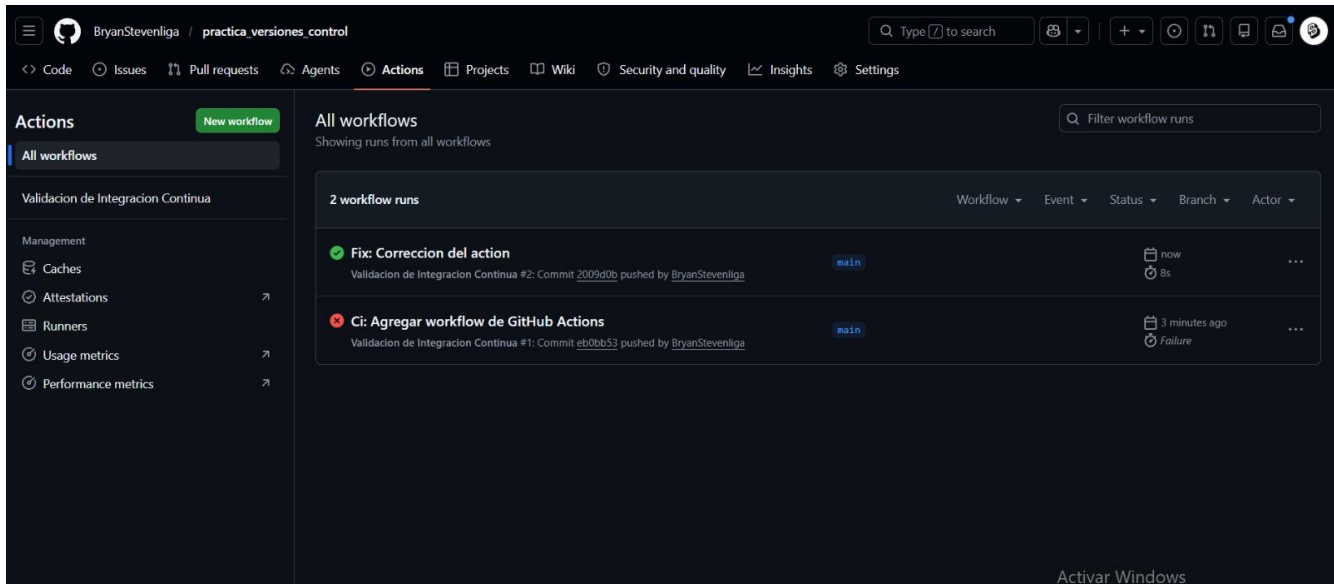
```
PS C:\Users\USER\Desktop\PRACTICA_DEVOPS\practica_versiones_control> git pull origin main
Updating 256b940..b200e22
Fast-forward
 index.html | 4 +++
 1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\USER\Desktop\PRACTICA_DEVOPS\practica_versiones_control> git add .
PS C:\Users\USER\Desktop\PRACTICA_DEVOPS\practica_versiones_control> git commit -m "Ci: Agregar workflow de GitHub Actions"
[main eb0bb53] Ci: Agregar workflow de GitHub Actions
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 .github/workflows/ci.yml
PS C:\Users\USER\Desktop\PRACTICA_DEVOPS\practica_versiones_control> git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (5/5), 436 bytes | 436.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/BryanStevenliga/practica_versiones_control.git
 b200e22..eb0bb53  main -> main
PS C:\Users\USER\Desktop\PRACTICA_DEVOPS\practica_versiones_control>
```

Supervisión y Corrección de Ejecuciones en GitHub Actions

17. **Actividad Realizada:** Se ingresó a la pestaña Actions del repositorio remoto para verificar el estado de las automatizaciones de la "Validacion de Integracion Continua". Tras detectarse un fallo inicial (Failure) en el primer commit (eb0bb53), se procedió a realizar un segundo envío con los ajustes necesarios bajo el commit (2009d0b) titulado "Fix: Correccion del action".

Resultado Obtenido: Como se detalla en la captura de pantalla, la plataforma procesó de manera exitosa la nueva ejecución en la rama main, cambiando el estado visual a

un check verde de conformidad (Status: Success). Esto confirmó que el pipeline de integración continua quedó corregido, funcional y ejecutándose en tan solo 8 segundos.



Historial de Commits y Estado Final del Repositorio

18. Actividad Realizada: Se procedió a realizar la auditoría final ingresando a los módulos de Commits y Pulse / Insights en GitHub para validar el cierre completo del proyecto bajo la rama principal (main). Se verificó la correcta secuenciación de los 11 commits generados colectivamente, registrando cada mensaje de cambio, autorías cruzadas, hashes de confirmación y estados de fusión.

Resultado Obtenido: Como se constata en las capturas de pantalla, el historial refleja un flujo de trabajo 100% integrado y exitoso. Se evidencia el procesamiento de los 4 Pull Requests completamente fusionados (Merged), la resolución definitiva del conflicto de código mediante el merge automático, y el estado final estable del entorno con el último ajuste automatizado de la acción (2009d0b) validado correctamente por el sistema.

Conclusiones

Steven Ninasunta: Trabajar en este entorno me enseñó a gestionar repositorios colaborativos desde su origen con el README.md, asumiendo el rol clave de autorizar los merges mediante la aprobación estricta de Pull Requests para proteger la rama principal main; gracias a este control de integración y al uso del archivo ci.yml, pude detectar a tiempo un fallo crítico en el pipeline y corregirlo con éxito a través de mi commit final 2009d0b.

Bryan Viracocha: La experiencia en este entorno colaborativo me aportó un aprendizaje clave sobre el rol de desarrollador en equipo, permitiéndome dominar la creación de ramas independientes para codificar estilos y maquetación, además de experimentar cómo se procesan y autorizan mis merges mediante Pull Requests; esta práctica me enseñó a resolver conflictos de código en Git y a validar que mis contribuciones pasaran con éxito las pruebas automatizadas del archivo ci.yml.

Investigación sobre Control de Versiones, Git y GitHub

1. Control de Versiones

Aspecto	Descripción
Definición	Sistema que registra los cambios en un archivo o conjunto de archivos a lo largo del tiempo para poder recuperar versiones específicas posteriormente
Utilidad	Permite saber quién escribió qué y cuándo, volver atrás en el tiempo, y recuperar cualquier versión anterior del proyecto sin perder nada
Ejemplo práctico	Un desarrollador modifica un archivo HTML: el VCS guarda la versión anterior, permite ver quién hizo el cambio en un día específico, y recuperar la versión de hace una semana si hay errores
Fuente	freeCodeCamp: "Aprende Git y control de versiones en una hora" ; SW Carpentry Git Novice

2. Git

Aspecto	Descripción
Definición	Sistema de control de versiones distribuido que rastrea el historial de cambios conforme personas y equipos colaboran en proyectos
Utilidad	Permite crear ramas para trabajar independientemente, fusionar cambios de forma inteligente, y mantener track de todos los Perubahan sin afectar a otros colaboradores
Ejemplo práctico	Un desarrollador crea una rama nueva-funcionalidad, hace cambios en archivos HTML/CSS independientemente, luego Git fusiona esos cambios automáticamente a la rama principal sin impactar el trabajo de otros
Fuente	GitHub Docs: "Acerca de Git" ; GitHub Docs: "About GitHub and Git"

3. GitHub

Aspecto	Descripción
Definición	Plataforma en la nube basada en Git para almacenar, compartir y colaborar en la escritura de código
Utilidad	Permite mostrar/compartir trabajo, trackear cambios, revisar código con otros, y colaborar en proyectos sin que los cambios afecten a colaboradores antes de integrar
Ejemplo práctico	Un estudiante sube su proyecto web HTML/MongoDB a GitHub, comparte el enlace con compañeros para revisión, y todos colaboran sin sobrescribir el trabajo ajeno
Fuente	GitHub Docs ; Wikipedia ; BenriosDev

4. Repositorio Local

Aspecto	Descripción
Definición	Copia integral del proyecto y su historial almacenada en tu computadora personal
Utilidad	Permite trabajar offline, hacer cambios sin conexión a internet, y sincronizar con el repositorio remoto cuando sea necesario
Ejemplo práctico	Creas un directorio, ejecutas git init para inicializar el repositorio local, trabajas en tus archivos HTML, haces commits localmente, luego conectas con GitHub usando git remote add
Fuente	GitHub Docs ; GitHub Gist: "Conexión a repositorios remotos"

5. Repositorio Remoto

Aspecto	Descripción
Definición	Copia del repositorio alojada en la nube (GitHub) accesible desde cualquier lugar

Aspecto	Descripción
Utilidad	Permite acceso desde cualquier dispositivo, colaboración en tiempo real, y backup seguro del código
Ejemplo práctico	Después de crear repositorio en GitHub con git remote add origin https://github.com/usuario/proyecto.git , subes cambios con git push origin main y tus compañeros pueden clonarlo
Fuente	GitHub Docs ; BenriosDev

6. Commit

Aspecto	Descripción
Definición	Confirmación de cambios que guarda un snapshot del proyecto en un punto específico del tiempo
Utilidad	Registra cambios con mensaje descriptivo, permite volver a versiones anteriores, y documenta el historial de modificaciones
Ejemplo práctico	Después de editar archivos HTML, usas git add . para staged, luego git commit -m "Agregué formulario de contacto" para guardar el cambio con mensaje
Fuente	GitHub Docs ; AprendeGIT ; Reddit learnprogramming

7. Branch (Rama)

Aspecto	Descripción
Definición	Línea de desarrollo independiente que se crea desde la rama principal para trabajar en funcionalidades sin afectar el código base

Aspecto	Descripción
Utilidad	Permite trabajar en características nuevas, experimentos, o correcciones sin impactar el proyecto principal hasta estar listo
Ejemplo práctico	Creas una rama con git checkout -b nueva-funcionalidad, trabajas en una nueva página HTML, haces commits, luego creas Pull Request para fusionar a main
Fuente	GitHub Docs ; AprendeGIT

8. Merge

Aspecto	Descripción
Definición	Fusión de cambios de una rama a otra, integrando el código desarrollado en una rama feature a la rama principal
Utilidad	Combina trabajo de diferentes desarrolladores, integra funcionalidades completas, y mantiene historial unificado del proyecto
Ejemplo práctico	En GitHub, vas a la pestaña Pull Request, haces clic en "Merge pull request" para fusionar tu rama feature a main, o usas git merge master en terminal
Fuente	Stack Overflow ; GitHub Docs: "About merge methods"

9. Pull Request

Aspecto	Descripción
Definición	Solicitud formal para integrar cambios de una rama a otra, permitiendo revisión y discusión antes de fusionar

Aspecto	Descripción
Utilidad	Facilita code review, discusión de cambios, automatización de pruebas, y approval workflows antes de mergear código
Ejemplo práctico	Desarrollador crea rama para nueva funcionalidad, hace commits, abre PR en GitHub, el equipo revisa cambios, hace comentarios, luego aprueba merge
Fuente	Instagram PR ejemplo ; BenriosDev ; AprendeGIT ; Reddit

10. Fork

Aspecto	Descripción
Definición	Copia de un repositorio de otro usuario en tu propia cuenta de GitHub
Utilidad	Permite contribuir a proyectos de otros sin acceso directo, trabajar en proyectos externos, y enviar cambios via Pull Request
Ejemplo práctico	Haces clic en "Fork" en un repositorio público, clonas tu fork localmente, haces cambios, creas Pull Request al repositorio original, el dueño acepta o rechaza
Fuente	BenriosDev ; Instagram Fork ; Adrián Perales

11. Conflictos de Integración

Aspecto	Descripción
Definición	Situación cuando dos desarrolladores modifican la misma línea de código en diferentes ramas y Git no puede fusionar automáticamente
Utilidad	Identificar y resolver conflictos evita errores en producción y asegura que cambios contradictorios se manejen manualmente

Aspecto	Descripción
Ejemplo práctico	Dos desarrolladores editan la misma línea de HTML en la misma sección, al hacer merge Git marca conflicto, debes usar herramienta como KDiff3 para resolver manualmente qué cambio mantener
Fuente	Git SCM ; Mentortay CI/CD

12. Integración Continua (CI)

Aspecto	Descripción
Definición	Práctica de confirmación de código periódica en repositorio compartido, integrando cambios frecuentemente
Utilidad	Detecta errores más rápido, reduce tiempo de debug, facilita fusión de cambios entre equipo, dedica más tiempo a escribir código y menos a resolver conflictos
Ejemplo práctico	Desarrollador hace push de código, GitHub Actions ejecuta pruebas automáticas (unitarias, integración, seguridad), si pasan el código se integra a main, si fallan se notifica
Fuente	GitHub Docs: "Integración continua" ; José Juan Sánchez CI/CD PDF

13. Entrega Continua (Continuous Delivery)

Aspecto	Descripción
Definición	Proceso de automatizar compilación, pruebas, configuración y implementación desde entorno de compilación a producción
Utilidad	Automatiza entrega de valor a usuarios finales sin errores, permite liberación continua, centra en feedback del cliente

Aspecto	Descripción
Ejemplo práctico	Después de que código pasa pruebas CI, se automatiza despliegue a entorno de prueba/preproducción, pero requiere aprobación manual para ir a producción
Fuente	Microsoft Learn Azure DevOps ; Ilimit CI/CD

14. Despliegue Continuo (Continuous Deployment)

Aspecto	Descripción
Definición	Automatización del despliegue a producción después de que código pasa todos los tests de CI, sin intervención manual
Utilidad	Código se despliega automáticamente en producción sin que el equipo haga nada, maximiza velocidad de entrega
Ejemplo práctico	Código pasa pruebas CI, GitHub Actions automáticamente despliega a producción sin aprobación manual, a diferencia de Continuous Delivery que requiere click final
Fuente	GitHub Docs: "Implementación continua" ; José Juan Sánchez HTML ; Instagram CICD

15. GitHub Actions

Aspecto	Descripción
Definición	Herramienta de automatización integrada en GitHub que permite crear workflows para compilar, testear y desplegar código
Utilidad	Reduce cadena de acciones para ejecución de código, reacciona automáticamente a eventos, crea flujos de CI/CD, compatible con cualquier lenguaje vía Docker

Aspecto	Descripción
Ejemplo práctico	Creas archivo .github/workflows/main.yml con workflow YAML que se ejecuta en cada push, instala Node.js, corre tests, y despliega si pasan todas las pruebas
Fuente	Plain Concepts ; Mentor Estech ; GitHub Actions Docs